

Brute-force using GPUs

Unleash the full potential of parallel processing in GPUs for cracking passwords

Jait Dixit and Mithun Mohandas
readersletters@thinkdigit.com

Forgetting passwords is quite a common occurrence and when this happens, either the files we protect are rendered worthless or sometimes a person's career can be on the line. In such cases, we end up typing every word/phrase of any relevance at all to the file and go through them one after the other. What we tried to do was a dictionary attack. When that fails, the last resort is to use every character known and use every combination and permute that. So if we have a 5-character password that might contain upper case letters, lower case letters and numbers, then we're faced with 776 million odd combinations that need to be checked. Every time you enter a password, you need to wait for the confirmation, then try another one until you've hit gold. This is exasperating. Here's where brute forcing software comes into the picture.

Passwords, aren't stored in any particular file. This practice was discarded long back as viewing the password-protected file within a hex-editor would simply reveal the password. So software manufacturers started hashing the passwords using various algorithms like MD4, MD5, SHA1, NTLM etc. And soon multiple hashing algorithms were applied

to complicate the process. So what we'll be doing now is extracting hash-keys from protected files and brute-forcing them to crack them. However, this is a tedious process and can take hours, months and even trillions of years depending on the complexity of the algorithm. A 6-character SHA1 hash can be cracked in under 10 minutes using a Core 2 Quad Extreme 9650, whereas a Radeon HD 5970 GPU would take less than five seconds! Thanks to parallel processing in the form of stream processors on AMD cards and CUDA cores on Nvidia cards.

Cain and Abel

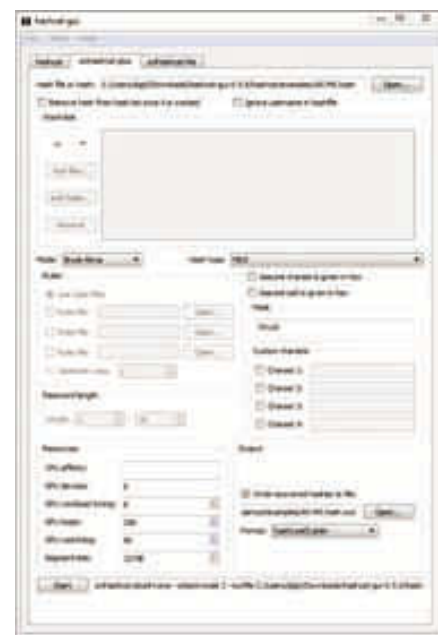
This tool only utilises the CPU, so we can only manage a fraction of what we can using a GPU. However, this tool deserves a worthy mention for the sheer number of protocols it supports.

ocl-hashcat plus

ocl-hashcat is a frequently updated tool that makes use of the parallel processing architecture present in modern day GPUs. We set ourselves to the task of cracking a Wi-Fi router's WPA2 passphrase. In order to achieve this, we set up our target router with a simple 8-character WPA2 passphrase. Then we setup a computer running BackTrack 5 R2 with a Wi-Fi card to capture packets which we'd later be working on.

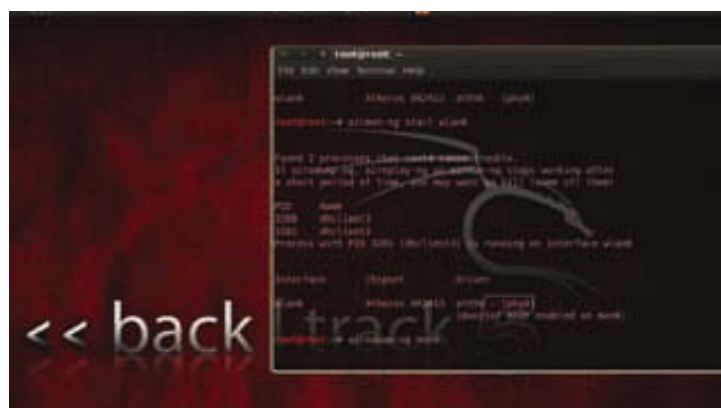
We began by using aircrack-ng. The first step was to run airmong and start monitoring the air-waves for a Wi-Fi signal. Then we started airodump-ng on an interface which works in the monitor mode, started

by airmong monitoring process to get the base station MAC address and address of clients connected to the same network. We observed the available networks to figure out which channel the



ocl-hashcat plus

router is operating on. Then we used airodump-ng to capture whatever incoming/outgoing packets we could from the router. During the dump process there had to be at least one four-way handshake (initial communication between the router-client) to take place on the air-waves. This could either be done actively or passively, in case of the active mechanism, the handshake was forced by using a ruse called fake de-authentication. While trying to crack a key passively, one had to wait until a client actually connected to the AP. Once that was done, we took the captured packets and used ocl-hashcat to perform a brute-force attack on them. This was performed on a rig comprising of Intel Core i7 Extreme 3960X, DX79SI, Radeon HD 5870. Needless to say, it didn't take long for us to guess the key. **d**



BackTrack 5 R2 – where majority of the work got done